C# PRACTICAL ASSIGNMENT · WEEK 02

Library Management System

Build a working console application to manage books and library members, borrow and return books, and handle common user input errors.

Language: C# **Platform: .NET Console** **Format: Individual task**

Overview

Build a **Library Management System** using a C# Console Application.

The application should allow users to manage books and library members, borrow and return books, and view information about the library.

Your goal is to build a working application with a clear user interface in the console and handle different situations correctly.

1 Main Menu

When the application starts, display a menu similar to the following:

```
================================
      LIBRARY MANAGEMENT
================================

1. Display All Books
2. Add New Book
3. Register New Member
4. Display All Members
5. Borrow a Book
6. Return a Book
7. Display Available Books
8. Display Borrowed Books
9. Exit

Choose an option:
```

The application should continue running until the user chooses **Exit**.

2 Book Management

The application should allow users to:

- Add a new book.
- Display all books.
- Display only available books.
- Display currently borrowed books.
- View the details of a specific book.

Each book should have information such as its ID, title, author, and availability status.

You can add other useful information if you think it is necessary.

3 Member Management

The application should allow users to:

- Register a new library member.
- Display all registered members.
- View the details of a specific member.

Each member should have a unique ID, name, and any other information you find useful.

4 Borrow a Book

A member should be able to borrow an available book.

Before completing the operation, the application should verify that:

- The requested book exists.
- The member exists.
- The book is available.

If the operation cannot be completed, display a clear message explaining why.

5 Return a Book

A member should be able to return a borrowed book.

The application should handle situations such as returning a book that is already available or requesting a book that does not exist.

6 Input Validation

The application should handle invalid user input without crashing.

Examples:

- Entering text instead of a number when selecting a menu option.
- Entering an invalid ID.
- Entering empty book or member names.
- Selecting an option that does not exist.
- Attempting to borrow an unavailable book.

7 Application Behavior

The application should maintain its data while it is running.

For example, when a user adds a book, that book should appear in the book list. When the book is borrowed, its availability should change. When it is returned, it should become available again.

> **Note:** Data persistence after closing the application is not required.

8 Additional Requirements

- Use C# and .NET Console Application.
- Keep the console interface clear and easy to use.
- Display appropriate success and error messages.
- Use meaningful names for variables, methods, and other program elements.
- Organize the code so that it is easy to understand and maintain.
- Avoid putting the entire application inside the `Main` method.

9 Bonus Challenges

These are optional and should only be attempted after completing the main requirements.

- Add the ability to search for a book by title.
- Prevent duplicate book and member IDs.
- Display the number of available and borrowed books.
- Add a borrowing limit for each member.
- Allow the user to cancel or repeat operations without restarting the application.

10 Submission Requirements

Submit the following:

1. The complete source code.
2. A README file explaining how to run the application.
3. Screenshots showing the main menu and important operations.
4. A short explanation of your implementation decisions and the challenges you faced.

Be prepared to explain your code and demonstrate that the application works correctly.

> **Important:** Focus on building a working solution and understanding your implementation. You are free to decide how to organize the application and solve the requirements.